

# 《燕园时轴物语》——Qt 大作业报告

## 一、程序功能介绍

### 1.1 项目概述

《燕园时轴物语》（Qt-Gaming）是一款基于 Qt 框架开发的 2D 像素风格校园历史主题角色扮演游戏。游戏以北京大学校史为背景，融合了“五四运动”、“西南联大”等历史元素，通过“时轴跃迁”的叙事设定，让玩家在不同的历史场景中探索、战斗、解谜。游戏采用 Tiled 等实现 2D 地图渲染，支持地图加载、角色控制、战斗系统、交互系统、粒子特效等完整游戏功能。

### 1.2 核心功能模块

#### 1.2.1 地图系统

游戏包含四张风格各异的地图：

| 地图名称 | 主题     | 核心玩法             |
|------|--------|------------------|
| 校史馆  | 旅程起点   | 初始训练、战斗教学        |
| 北大红楼 | 新文化运动  | 迷宫探索、历史文献交互      |
| 西南联大 | 战火中的教育 | 躲避轰炸、收集碎片        |
| 未名湖  | 燕园明珠   | 火焰地形寻宝、最终 boss 战 |

地图数据采用 Tiled 编辑器制作，导出为 JSON 格式，由 MapLoader 类解析加载。每张地图均包含多层结构（地板、墙壁、装饰、交互物等），通过图层叠加实现丰富的视觉层次。

#### 1.2.2 战斗系统

战斗系统是游戏的核心玩法之一，包含以下要素：

- 七种技能：**I（天火燎原）、J（八面炼狱）、K（瞬影浪斩）、N（月牙八刃）、H（熔岩弹）、L（玄武盾）、O（加速）
- 技能消耗 MP：**I 技能消耗 10 MP，K 技能消耗 15 MP，L 盾牌消耗 5 MP，随等级提升解锁强化形态
- 敌人生成：**巢穴定时生成，随机游走 AI，三方向射击
- 打击反馈：**受伤定身、爆炸特效、血条动态变化

#### 1.2.3 交互系统

交互系统为游戏提供了丰富的探索体验：

- 宝箱 & 门：**收集钥匙开启门区域（BFS 连通区域移除）

- **历史文献阅读：**靠近交互物按 R 键，弹出历史文献对话框
- **可收集物品：**红/蓝/紫钻石，分别恢复 HP/MP/攻击翻倍

#### 1.2.4 视觉与特效

- **墙体阴影：**根据下方/左侧/左下角邻居自动选择阴影瓦片
- **粒子系统：**梅花瓣飘落、暴风雪、爆炸动画
- **Z 值分层：**草(-4) → 水(-3) → 路(-2) → 墙(-1) → 装饰(0) → 玩家(2) → 特效(50+) → HUD(1000+)

## 二、项目各模块与类设计细节

### 2.1 整体架构

#### Qt-Gaming

|                                |                        |
|--------------------------------|------------------------|
| —— main.cpp                    | → 程序入口，创建 QApplication |
| —— Game (game.h/cpp)           | → 游戏主控类 (~5000 行)      |
| —— Player (player.h/cpp)       | → 玩家类                  |
| —— TileMap (tilemap.h/cpp)     | → 地图管理类                |
| —— Tile (tile.h/cpp)           | → 瓦片基类                 |
| —— MapLoader (maploader.h/cpp) | → Tiled JSON 解析器       |
| —— Enemy (enemy.h/cpp)         | → 敌人类 + 怪物炮弹           |
| —— Skill (skill.h/cpp)         | → 技能投射物 (6 种)          |
| —— Pet (pet.h/cpp)             | → 宠物跟随                 |
| —— Spawner (spawner.h/cpp)     | → 怪物巢穴                 |

### 2.2 核心类详解

#### 2.2.1 Game 类

游戏主控类，负责窗口管理、游戏循环、地图加载、键盘交互、技能系统、HUD 系统、传送门、菜单系统等全部核心功能。

#### 关键数据结构：

##### • 投射物容器：

```

    QVector<Projectile*> projectiles;           // I 技能火球
    QVector<BladeWave*> bladeWaves;           // K 技能刀浪
    QVector<Enemy*> enemies;                   // 所有敌人
  
```

##### • 交互系统：

```

    QVector<Tile*> chests;                      // 宝箱列表
    QVector<Tile*> doors;                      // 门列表
    float keyCount;                          // 钥匙计数
  
```

- 地形:

`QVector<QRectF> fireRects;` // 火焰区域 (碰撞检测)

- 子空间区域 (校史馆地图):

`int subspaceId;` 等等。

## 2.2.2 Player 类

玩家角色控制, 核心功能包括:

- GIF 多状态动画: Idle/Run/Enhanced/Cast, 支持朝向左/右/上/下
- 碰撞检测: `hitboxRect()` 返回右下角 32×32 碰撞框
- 属性系统: HP/MP/EXP/Level (1-20 级), 升级回满血量
- 跨地图状态保持: `restoreState()` 保存/恢复等级、HP、MP、形态

## 2.2.3 TileMap 类

地图碰撞管理, 采用空间索引网格:

- 空间索引网格: `gridPixelSize = tileSize * GRID_SIZE (256px)`, 0(1) 碰撞查询
- 墙壁碰撞: `collidesWithWall(QRectF)`, 支持添加/移除墙壁瓦片
- 水碰撞: 独立索引 `gridWaters`, 仅阻挡玩家不阻挡子弹
- 火焰碰撞: `collidesWithFire(QRectF)`, 用于地形伤害判定

## 2.2.4 MapLoader 类

Tiled JSON 解析器, 其解析流程为: `QFile` → `QJsonDocument` → 读取 `tilesets` 建立 GID→图片映射 → 遍历 `layers` 存储数据。

## 2.2.5 Enemy 类

敌人包含以下机制:

- 8 种 GIF 外观: `monster_bomb.gif` 等随机选取
- 随机游走: 每 60 帧换方向, 撞墙回退
- 三方向射击: 从 8 方向随机选 3 个发射紫色炮弹
- 攻击间隔动态调整: 随玩家等级缩短 (180 → 60 帧)
- 血条显示: 头顶 32×4 红色血条

## 2.2.6 Skill 系统

游戏包含 6 种投射物类型:

| 类名         | 对应技能        | 特点               |
|------------|-------------|------------------|
| Projectile | I 技能 (3 级+) | GIF 火球, 8 方向自动旋转 |

| 类名                 | 对应技能        | 特点       |
|--------------------|-------------|----------|
| SimpleProjectile   | I 技能（1-2 级） | 红色椭圆弹    |
| BlueProjectile     | N 技能        | 月牙形，8 方向 |
| TriangleProjectile | H 技能        | 三角形破空梭   |
| BladeWave          | K 技能（1 级）   | 矩形刀浪     |
| DaolangWave        | K 技能（2 级+）  | GIF 刀浪   |

所有 GIF 在启动时预加载到全局缓存，运行时零解码开销。

### 三、小组成员分工情况

事实上，我们有明确的分工：

| 成员  | 主要负责模块    | 具体工作内容                                          |
|-----|-----------|-------------------------------------------------|
| 陈锐力 | 地图、美术与后期  | Tiled 地图制作，图块绘制，玩法构思，主菜单美术设计，交互系统，后期视频剪辑与作业报告撰写 |
| 陈治中 | 战斗、细节     | 战斗技能设计与实现，角色与怪物设计，部分玩法实现与细节优化                   |
| 叶恒锋 | 文案写作与整合处理 | github 整合，菜单功能实现                                |

### 四、项目总结与反思

#### 4.1 项目成果

本项目成功实现了一款以北大校史为主题的 2D 像素风 RPG 游戏，包含：

- **完整的游戏循环：**60FPS 流畅运行，支持地图切换、状态保持
- **四张风格各异的地图：**校史馆、北大红楼、西南联大、未名湖
- **七种技能与完整战斗系统：**近战/远程/防御/爆发，支持等级成长
- **丰富的交互机制：**宝箱、门、钥匙、历史文献阅读
- **精致的视觉呈现：**随机纹理、边缘判定、粒子特效、墙体阴影
- **历史叙事融合：**五四运动、西南联大等历史元素融入玩法

#### 4.2 技术亮点

- **空间索引碰撞检测：**0(1) 时间复杂度，支持大量瓦片
- **帧缓存预加载机制：**所有 GIF 解码一次，运行时零开销

- **跨地图状态保持**: 等级/HP/MP/形态在不同地图间无缝传递
- **Z 值分层渲染**: 20+ 层级, 保证正确的遮挡关系
- **Tiled 地图深度集成**: 支持图层、对象层、自定义属性
- **BFS 门区域移除**: 连接门区域整片消失
- **边缘判定自动纹理**: 水/墙根据邻居自动选择纹理

### 4.3 反思与改进方向

值得肯定的:

- 团队协作良好, 分工明确, 代码整合顺利
- 项目规划合理, 功能迭代有序
- 历史与游戏融合自然, 主题鲜明
- 最后剪辑出的视频很有艺术效果

可改进的:

- 代码结构: Game 类约 5000 行, 可进一步拆分 (如将技能系统独立为 SkillManager)
- 资源管理: 部分图片资源未使用, 可清理
- 音效系统: 当前无音效, 可增加背景音乐和技能音效
- 存档系统: 当前无法保存游戏进度, 可增加 QSettings 存档
- 性能优化: 大量粒子特效时可进一步优化渲染

### 4.4 我们学到了什么?

这次大作业让我们在**技术与团队协作**两个维度都获得了宝贵的成长。

#### 4.4.1 Qt 框架与 C++ 实践

通过本次项目, 我们深入实践了 Qt 框架的核心机制。在图形渲染方面, 我们熟练运用了 QGraphicsView/QGraphicsScene/QGraphicsItem 架构, 理解了 2D 游戏的渲染管线与 Z 值分层原理。在事件处理方面, 我们掌握了键盘事件、鼠标事件和定时器驱动的游戏循环模式, 实现了 60FPS 的流畅交互。在资源管理方面, 我们学习了 QPixmap 与 QMovie 的 GIF 帧提取与缓存机制, 通过预加载所有动画帧, 将运行时解码开销降为零, 保证了战斗场景的流畅度。此外, 我们还深入使用了 Qt 的 JSON 解析模块(QJsonDocument)和资源系统(.qrc), 构建了完整的数据驱动地图加载流程。

#### 4.4.2 Git 与团队协作

在版本控制方面, 我们从零开始学习了 Git 的核心工作流。从一开始配置 SSH 密钥、克隆仓库, 到后续的 git pull 同步队友更新、git commit 记录修改、解决合并冲突, 我们完整地经历了一个真实项目的协作流程。通过这次实践, 我们理解到版本控制不仅是代码管理工具, 更是团队沟通的基础设施——清晰的 commit 信息、及时的 pull 同步、合理的分工避免冲突, 这些习惯让我们在多人协作中始终保持高效。

#### 4.4.3 Tiled 地图编辑器与工作流集成

Tiled 的学习曲线比我们预想的更陡, 但也带来了最大的成就感。我们学会了创建瓦片集、设计多层地图 (地面层/装饰层/碰撞层)、配置对象层 (传送门/玩家起点)。更重要

的是，我们建立了完整的工作流：Tiled 设计地图 → 导出 JSON → Qt 程序解析 → 游戏内渲染。这个流程让我们意识到，游戏开发不仅是写代码，更是多个工具链的集成与协作。

#### 4.4.4 团队协作的感悟

作为三人团队，我们在这次项目中形成了“一人主战斗、一人主地图、一人主细节”的分工模式。每周的进度同步、代码合并时的沟通、遇到 Bug 时的共同排查，都让我们体会到团队协作的默契与力量。尤其是在项目后期，我们学会了合理取舍——先保证核心功能的稳定性，再追求边缘效果的精致度。